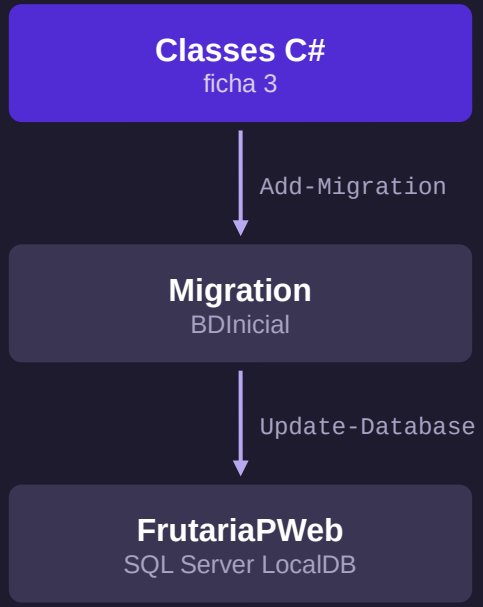
PROGRAMAÇÃO WEB · 2026/27 · AULA LABORATORIAL

Ficha 4
Base de Dados

Dois comandos transformam as classes da ficha 3 na base de dados FrutariaPWeb. Depois confirma-se, tabela a tabela, que ficou como as classes descrevem.



PLANO DA AULA

Como lá vamos chegar

1

Ponto de partida

O projeto da ficha 3 compila, mas ainda não arranca.

2

Gerar a migration

Alínea 3.a. Um comando escreve a migration.

3

Criar a base de dados

Alínea 3.b. Outro comando cria a FrutariaPWeb.

4

Verificar as tabelas

Alíneas 3.c e 3.d. Coluna a coluna, contra as classes.

5

Primeiro arranque

Complemento. A aplicação cria os roles e o administrador.

Como usar os materiais: estes slides explicam o que cada comando faz e como ler o resultado. O guia HTML da ficha 4 tem o passo a passo, com o que escrever, onde, e o que verificar.

PONTO DE PARTIDA

O que a ficha 3 deixou, e o que falta

Já existe, em C#

- Três entidades na pasta `Entities`
- `ApplicationDbContext` com três `DbSet`
- `ApplicationUser` com os campos da loja
- Connection string para `FrutariaPWeb`
- `Program.cs` com os dados iniciais

Ainda não existe

- A base de dados `FrutariaPWeb`
- As tabelas e as colunas
- Os roles e o administrador

O que o F5 mostra desde a alínea 3.k da ficha 3

```
SqlException: Cannot open database "FrutariaPWeb" requested by the login. The login failed.
```

Hoje: a base de dados nasce das classes. Não se escreve SQL nem se cria nenhuma tabela à mão.

CONTEXTO

Dois caminhos na ficha 3, a mesma base de dados

Projeto criado para SQL Server LocalDB

`Data/Migrations/`

`00000000000000_CreateIdentitySchema`
do template: as tabelas do Identity

`<data>_BDInicial`
nova: colunas do ApplicationUser e as 3 tabelas da loja

Projeto criado para SQLite

`Migrations/  (pasta nova, na raiz do projeto)`

`<data>_BDInicial`
nova: todas as tabelas, as do Identity e as da loja

A migration do template foi apagada na alínea 3.e (passo 4).

FrutariaPWeb
11 tabelas, iguais nos dois casos

Em qual estamos? No Gerenciador de Soluções, ver se existe `Data/Migrations/00000000000000_CreateIdentitySchema.cs`. Se existir, é o caminho da esquerda.

ALÍNEA 3.a · A CONSOLA

O Console do Gerenciador de Pacotes

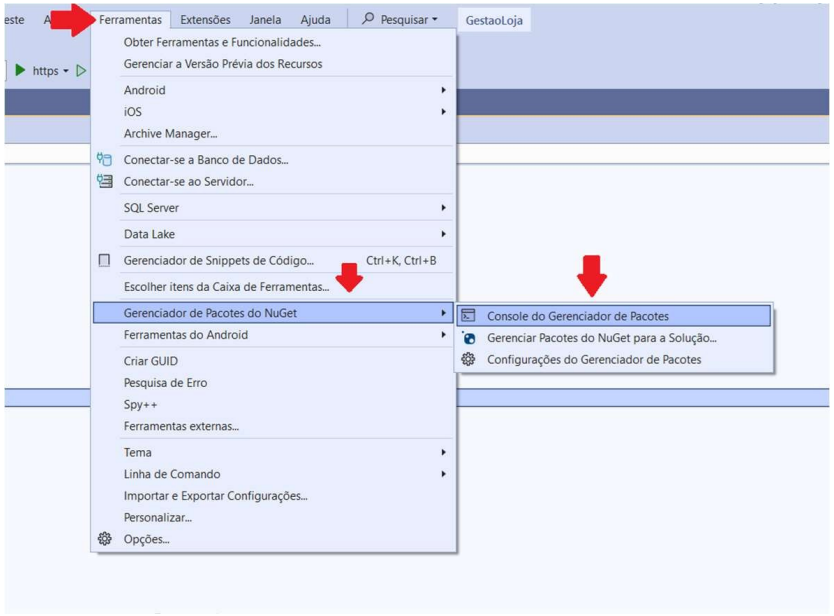


Figura 1 da ficha: Ferramentas > Gerenciador de Pacotes do NuGet > Console do Gerenciador de Pacotes.

Uma consola dentro do Visual Studio

Uma consola PowerShell. Escreve-se o comando à frente de `PM>` e carrega-se em Enter.

De onde vêm os comandos

`Add-Migration`, `Update-Database`, `Remove-Migration` e `Drop-Database` vêm do pacote `Microsoft.EntityFrameworkCore.Tools`, que o template com Contas Individuais inclui.

Projeto padrão

A lista no topo da consola diz sobre que projeto atuam os comandos. Aqui só há um: `GestaoLoja`.

ALÍNEA 3.a · ADD-MIGRATION

Add-Migration: o que acontece ao carregar em Enter

1. Compila

Se o projeto tiver erros, pára aqui com `Build failed.`

2. Constrói o modelo

Lê o `ApplicationDbContext`: os `DbSet`, as entidades, o `ApplicationUser` e o Identity.

3. Compara

Com a fotografia da última migration (o snapshot), ou com nada, se for a primeira.

4. Escreve

A diferença em C#: a migration, o `.Designer.cs` e o snapshot novo.

Na consola

```
PM> Add-Migration BDInicial
Build started...
Build succeeded.
To undo this action, use Remove-Migration.
```

Não toca na base de dados

A `FrutariaPWeb` continua a não existir. Só aparecem ficheiros no projeto.

Os dados iniciais não correm

O comando corre o `Program.cs` só até ao `Build()`. Por isso funciona sem base de dados.

ALÍNEA 3.a · O FICHEIRO GERADO

Da classe Produto ao CreateTable

Entities/Produto.cs (excerto)

```
[StringLength(100)]
[Required]
public string? Nome { get; set; }
public string? Origem { get; set; }
public decimal EmStock { get; set; }
public int CategoriaId { get; set; }
public int? ModoEntregaId { get; set; }
```

Add-Migration →

<data>_BDInicial.cs (excerto do método Up)

```
Nome = table.Column<string>(
    type: "nvarchar(100)", nullable: false),
Origem = table.Column<string>(
    type: "nvarchar(max)", nullable: true),
EmStock = ...(type: "decimal(18,2)", ...),
CategoriaId = ...(type: "int", nullable: false),
ModoEntregaId = ...(type: "int", nullable: true)
```

O ? não manda na coluna

`Nome` é `string?`, mas o `[Required]` torna a coluna `NOT NULL`. O `?` é só para o compilador.

Sem anotações, sem limites

`Origem` fica `nvarchar(max)` e aceita nulos. `EmStock` fica `decimal(18,2)`, o tipo por omissão.

int ou int?

`CategoriaId` sem nulos: relação obrigatória. `ModoEntregaId` com nulos: relação opcional.

ALÍNEA 3.b · UPDATE-DATABASE

Update-Database: das migrations à base de dados

1. **Lê a connection string**
   A `DefaultConnection` do `appsettings.json`.

2. **Cria a base de dados**
   No `(localdb)\MSSQLLocalDB`, porque ainda não existe.

3. **Vê o que falta**
   Compara as migrations com a `__EFMigrationsHistory`.

4. **Aplica e regista**
   Executa as que faltam, por ordem, e regista cada uma.

Na consola (projeto criado para SQL Server LocalDB)

```
PM> Update-Database
Build started...
Build succeeded.
Applying migration '00000000000000_CreateIdentitySchema'.
Applying migration '20261028101532_BDInicial'.
Done.
```

Depois: a tabela `__EFMigrationsHistory`

MigrationId	ProductVersion
`00000000000000_CreateIdentitySchema`	`8.0.x`
`20261028101532_BDInicial`	`8.0.x`

Pode correr-se outra vez

Sem migrations por aplicar, não faz nada: `No migrations were applied.`

Onde ficam os dados

Em `FrutariaPWeb.mdf` e `_log.ldf`, na pasta do utilizador. Quem os gere é o LocalDB.

ALÍNEA 3.c · VER A BASE DE DADOS

Onde ver a base de dados no Visual Studio

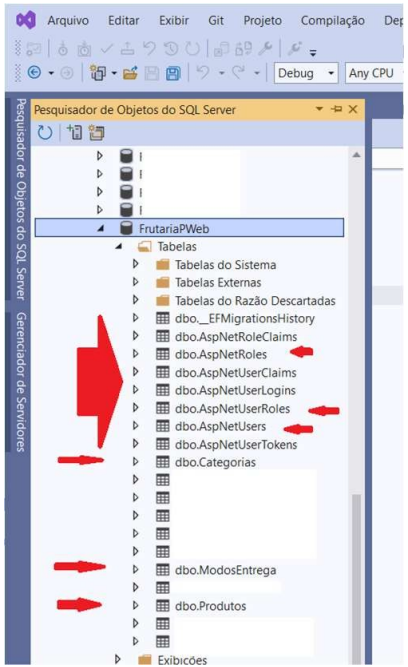


Figura 5 da ficha

Exibir > Pesquisador de Objetos do SQL Server

↓

SQL Server

↓

(localdb)\MSSQLLocalDB

↓

Bancos de Dados

↓

FrutariaPWeb

↓

Tabelas

`Ctrl+\, Ctrl+S`

11 tabelas

1 `__EFMigrationsHistory`: do Entity Framework

7 `AspNet...`: do Identity

3 `Categorias, ModosEntrega, Produtos`: os nossos DbSet

A ficha diz Server Explorer

As figuras são do Pesquisador de Objetos do SQL Server, duas linhas abaixo no mesmo menu.

As três tabelas da loja, vistas da base de dados

Categorias

```
Id  int  PK
Nome  nvarchar(max) · nulo
Ordem  int
UrlImagem  nvarchar(max) · nulo
Imagem  varbinary · nulo
```

1 N

Produtos

```
Id  int  PK
Nome  nvarchar(100)
Detalhe  nvarchar(200)
UrlImagem  nvarchar(200) · nulo
Imagem  varbinary · nulo
Preco  decimal(10,2)
Promocao, MaisVendido  bit
EmStock  decimal(18,2)
Disponivel  bit
Origem  nvarchar(max) · nulo
CategoriaId  int  FK
ModoEntregaId  int  FK · nulo
```

N 0..1

ModosEntrega

```
Id  int  PK
Nome  nvarchar(100)
Detalhe  nvarchar(200) · nulo
```

· nulo = a coluna aceita NULL

PK: chave primária FK: chave estrangeira

Chave primária

`Id` em cada tabela, numerado pela base de dados (`IDENTITY(1,1)`).

CategoriaId: em cascata

Não aceita nulos. Apagar uma categoria apaga os seus produtos.

ModoEntregaId: sem cascata

Aceita nulos. Apagar um modo de entrega não apaga os produtos.

ALÍNEA 3.d · AS FIGURAS DA FICHA

Onde o resultado difere das Figuras 6 e 7

Tabela	Na figura da ficha	No nosso projeto
Produtos	`Titulo nvarchar(MAX)`	Não existe: a classe não tem `Titulo`
Produtos	`Origem nvarchar(200)`	`Origem nvarchar(MAX)`: sem `[StringLength]`
AspNetUsers	`Nome`, `Apelido nvarchar(MAX)`	`nvarchar(100)`: `[StringLength(100)]`
AspNetUsers	`DataNascimento datetime2`	Não existe
AspNetUsers	`NIF int`	`NIF bigint`: a propriedade é `long?`
AspNetUsers	`Rua`, `Localidade1`, `Localidade2`, `Pais nchar(40)`	`Rua`, `Localidade`, `Cidade`, `Pais nvarchar(100)`
AspNetUsers	Não aparecem	`CodigoPostal`, `Telemovel`, `Estado`, `DataRegisto`

A referência é a classe, não a figura

Não acrescentar `Titulo` nem `DataNascimento` às classes para imitar as figuras.

ALÍNEA 3.d · IDENTITY

As tabelas do Identity: utilizadores e roles



Um utilizador pode ter vários roles, e um role vários utilizadores: a AspNetUserRoles liga os dois.

As outras quatro	Para que servem
`AspNetUserClaims`	Informações extra sobre um utilizador
`AspNetUserLogins`	Logins externos (Google, Microsoft)
`AspNetUserTokens`	Tokens, por exemplo da autenticação de dois fatores
`AspNetRoleClaims`	Informações extra sobre um role

Vêm com o Identity. Nesta fase ficam vazias.

Estado: sem ? nem [Required]

`string Estado`, sem `?`, fica `NOT NULL`. Com o `Nullable` ativo no projeto, o EF lê o `?`: sem ele, a coluna é obrigatória.

COMPLEMENTO · PRIMEIRO ARRANQUE

Primeiro arranque: os dados iniciais

AspNetRoles

Name	NormalizedName
Admin	ADMIN
Gestor	GESTOR
Cliente	CLIENTE

AspNetUsers

UserName	Nome	EmailConfirmed	Estado
admin@localhost.com	Administrador	True	Pendente

AspNetUserRoles

UserId	RoleId
Id do administrador	Id do role Admin

No Pesquisador de Objetos: clique direito na tabela > Exibir Dados (View Data).

F5: a aplicação volta a abrir

O `Program.cs` chama a `Inicializacao` antes de atender pedidos.

Hash, não palavra-passe

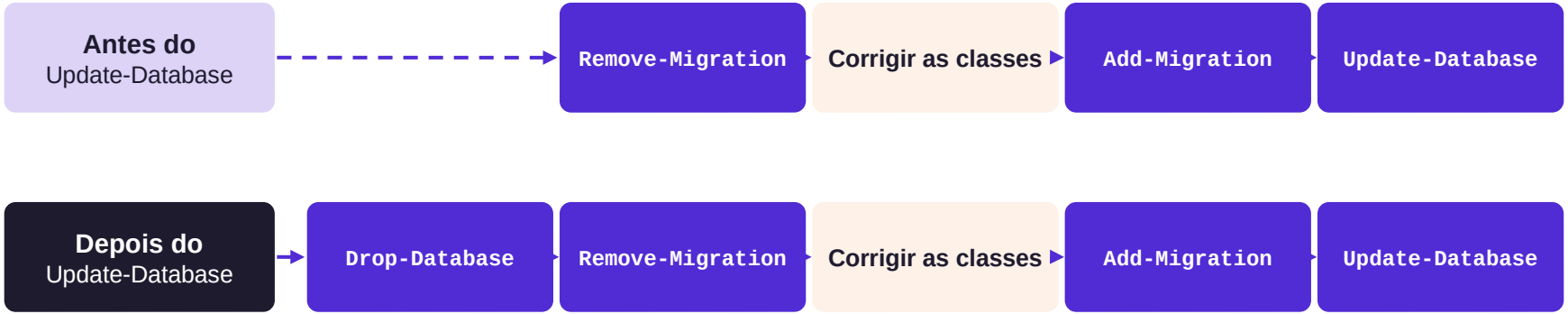
A coluna `PasswordHash` não guarda `Is3C..00`: guarda o hash dela.

Segundo F5: nada de novo

O método só cria o que não existe. As linhas ficam as mesmas.

QUANDO ALGO FALHA · CORRIGIR

Corrigir a base de dados: sempre pelas classes



A única diferença: com a base de dados já criada, apaga-se primeiro com `Drop-Database`.

Porque não apagar os ficheiros à mão

O `Remove-Migration` também repõe o snapshot. Com os ficheiros apagados à mão, o snapshot ficava com o modelo errado e a migration seguinte saía incompleta.

O que não fazer

- Alterar colunas no designer e carregar em Atualizar
- Editar os ficheiros das migrations
- Copiar para as classes as colunas das Figuras 6 e 7

QUANDO ALGO FALHA

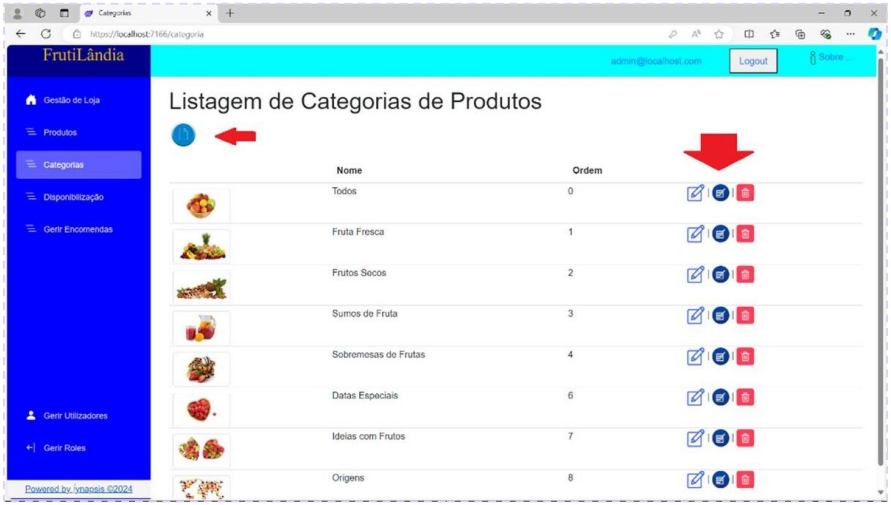
Erros frequentes

Sintoma	Causa e solução
"Add-Migration" não é reconhecido	Falta o pacote Microsoft.EntityFrameworkCore.Tools. Instalar com a versão dos outros pacotes do EF.
Build failed.	O projeto tem erros de compilação. Corrigir e repetir o comando.
Aviso sobre a propriedade decimal EmStock	Esperado: decimal(18,2) por omissão. Pode ignorar-se.
Método Up vazio, ou sem as tabelas da loja	Faltam os DbSet (ficha 3, alínea 3.c). Remove-Migration, corrigir, Add-Migration.
error: 50 ou Local Database Runtime error	O LocalDB não arrancou ou não está instalado. Repetir; sqllocaldb start MSSQLLocalDB.
There is already an object named AspNetRoles	A base de dados já tinha tabelas. Drop-Database e Update-Database.
Remove-Migration: has already been applied	A migration já está na base de dados. Drop-Database primeiro.
FrutariaPWeb não aparece no Pesquisador	Atualizar a lista. Confirmar Database=FrutariaPWeb no appsettings.json.
F5: Cannot open database "FrutariaPWeb"	O Update-Database não correu, ou falhou.

A SEGUIR

O que vem a seguir: ficha 5

Ficha 5: as páginas



Listagem de categorias, no fim da ficha 5 (figura da ficha 5).

O que se faz

- Home, Sobre, layout e menu
- As páginas de gestão de categorias, produtos e modos de entrega, geradas pelo Visual Studio a partir das classes
- Login como administrador e os primeiros registos

O que já está pronto

As tabelas de hoje e o administrador do primeiro arranque. A ficha 5 só lhes junta as páginas.

PARA LEVAR DA AULA

Das classes às tabelas

Add-Migration

Compara as classes com o snapshot e escreve a diferença em C#. Não mexe na base de dados.

Update-Database

Aplica as migrations em falta e regista-as na __EFMigrationsHistory.

Classe e tabela

Cada DbSet dá uma tabela, cada propriedade uma coluna, cada relação uma chave estrangeira.

Nulos

O [Required], ou um tipo sem ?, dá NOT NULL. O ? de string? é só para o compilador.

Identity

Sete tabelas AspNet sem escrevermos classes. O ApplicationUser junta colunas à AspNetUsers.

Corrigir

Pelas classes e com uma migration, nunca à mão na base de dados.